ICCS205 – Numerical Computation

Justin Copeland

1 | Introduction and Root Finding

Introduction

In this class, we will learn how to do math with computers (not exactly the same thing as running `2+2` on the Python interpreter, mind you). As an example, suppose that we wanted to solve the following equation:

$$x^2 - 2 = 2$$

Solving this is easy – add two to both sides then take the square root and we find that $x = 2$. But, how might we get a computer to solve this? Consider a less trivial equation:

$$e^x = x$$

Attempting to solve it might go something like this:

$$\begin{aligned} e^x &= x \\ \ln(e^x) &= \ln x \\ x\ln(e) &= \ln x \\ x &= \ln x \end{aligned}$$

Which is yet another harder equation. Worry not, for we can use computers to help us solve them! We will also explore other things like solving

- Systems of linear equations
- Integration problems
- Optimization problems
- And more

Class logistics

Item	Score
Midterm	40%
Final	40%
Homework	10%
Project	10%

- There are seven sets of homework
- Project can be 1-2 people. No fixed topic. Just "do something cool that you never thought computers could do before"

Root finding

Imagine that we would like to solve an equation that looks like

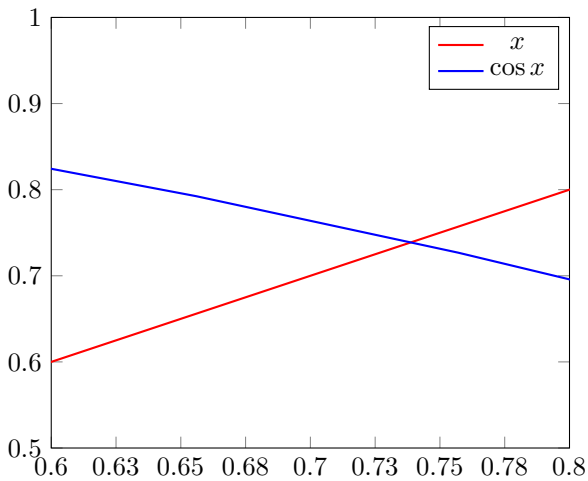
$$x = \cos(x)$$

Our goal is to find some value of x that makes both sides equate. However, this is a transcendental equation – we cannot solve it by hand.

$$\begin{aligned} x &= \cos(x) \\ \arccos(x) &= \arccos(\cos(x)) \\ \arccos(x) &= x \\ \cos(\arccos(x)) &= \cos x \\ &\vdots \end{aligned}$$

Let's try plotting x against $\cos x$ and see what happens.

```python
import numpy as np
from matplotlib import pyplot as plt

plt.figure(figsize=(20, 8))

xs = np.linspace(0, 2, 100)
y1s = xs
y2s = np.cos(xs)

plt.plot(xs, y1s, label='y=x')
plt.plot(xs, y2s, label='y=cos(x)')

plt.xlim(0.6, 0.8)
plt.ylim(0.5, 1)
plt.grid()
plt.legend()
```



We can see that it looks like the solution is around `0.7375`:

$$\cos(0.7375) = 0.7401$$

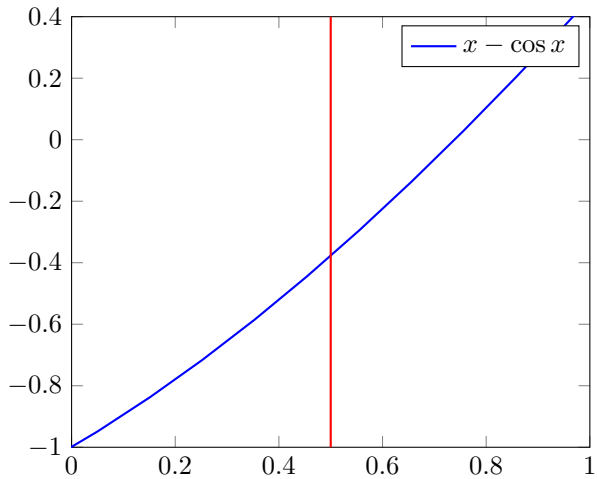
We're close, but not quite there yet. Instead of doing it manually, let's get a computer to do it. We could loop and find a spot where $x = \cos x$ but it's going to take a ton of iterations. So, what's the solution when linear search is slow...? *Binary Search.*

Bisection

What if I told you, that every equation in the world can be turned into $f(x) = 0$.

$$x = \cos x$$
$$x - \cos x = 0$$

And, not only can we do this, but we can use it to solve *any equation in the world.* How do we do this? First, we start with two bounds. Then, we make a guess in the middle.



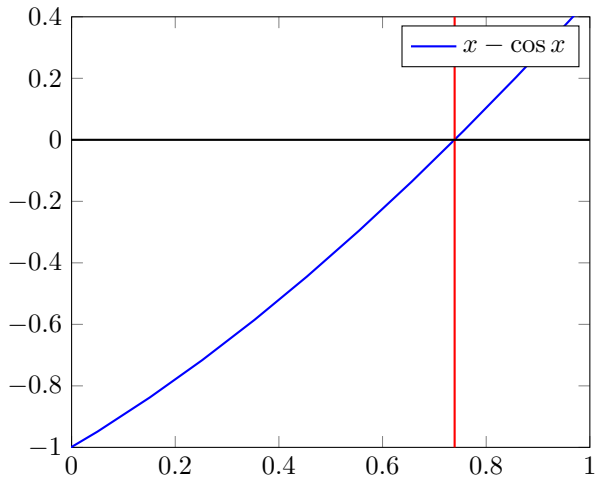
With each iteration, we will be throwing away a side, and in doing so, halving our uncertainty. In this case, we know that we need to discard the left side since our answer is to the right, but how do we determine which side to discard in general?

- We want to pick sides where the y-bounds have different signs, because, through continuity, the curve must pass zero somewhere
- If we start bounds with different signs, we know that there must be a bound between them.

Using this algorithm – known as the **Bisection Method** – we can solve `f(x) = 0` functions.

```python
import numpy as np
from matplotlib import pyplot as plt
from dataclasses import dataclass

@dataclass
class BisectionResult:
    midpoint: float
    uncertainty: float

# Return midpoint and uncertainty
def bisection(f, l: float, r: float, n: int) -> BisectionResult:
    # guess, pick side, repeat
    midpoint: float = (l + r) / 2
```

```
    if f(midpoint) == 0 or n == 0:
        return BisectionResult(midpoint=midpoint, uncertainty=(r-l)/2)
    else:
        if f(l) * f(midpoint) < 0:
            return bisection(f, l, midpoint, n - 1)
        elif f(r) * f(midpoint) < 0:
            return bisection(f, midpoint, r, n - 1)

def f(x):
    return x - np.cos(x)

sol = bisection(f, 0, 1, 20)
print(sol)
print(sol.midpoint, np.cos(sol.midpoint))

xs = np.linspace(0, 1, 100)
plt.plot(xs, f(xs), color='red')
plt.axvline(sol.midpoint, color='blue')
plt.axhline(0, color='black')
```



Can we do better?

Newton's method

What would be a better guess than the middle? If we know the slope and a starting point, we can determine the intersection. How do we get slope – derivatives.

- We know that lines can be represented as $y = mx + b$
- If we guess a point x_0, its y-coordinate is $f(x_0)$.
- If we know the slope, and that the curve passes through our point $(x_0, f(x_0))$, we can solve for b

$$\begin{aligned} f(x_0) &= f'(x_0)x_0 + b \\ b &= f(x_0) - f'(x_0)x_0 \end{aligned}$$

To solve for the next guess, we set $y = 0$ and then solve.

$$\begin{aligned} y &= mx_1 + c \\ 0 &= mx_1 + c \\ x_1 &= \frac{-c}{m} \end{aligned}$$

So:

$$\begin{aligned} x_1 &= \frac{f'(x_0)x_0 - f(x_0)}{f'(x_0)} \\ x_1 &= x_0 - \frac{f(x_0)}{f'(x_0)} \end{aligned}$$

The formula above is known as the *update rule* – a way to get a better answer given an answer.

$$x_n = x_n - \frac{f(x_n)}{f'(x_n)} = x_{n+1}$$

Let's try it out. Suppose we wanted to solve

$$f(x) = x^3 - x - 1$$

```python
def f(x):
    return x**3 - x - 1

def fp(x):
    return 3*x**2 - 1

def newton(f, fp, x0: float, n: int) -> float:
    guess = x0
    for i in range(n):
        guess = guess - f(guess) / fp(guess)
    return guess

newton(f, fp, 2, 5)
```

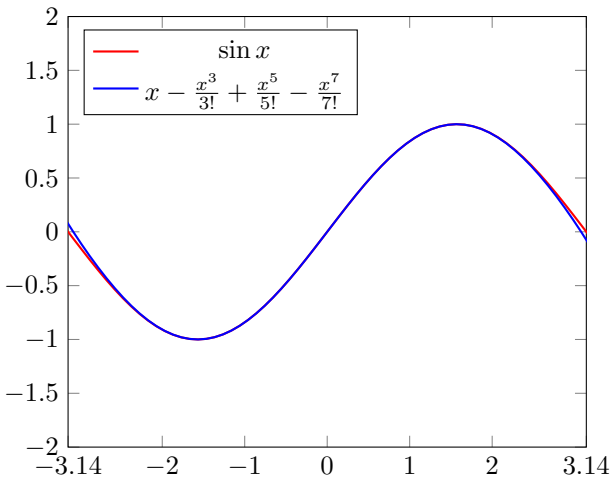
2 | Taylor Series and Derivatives

Taylor Series

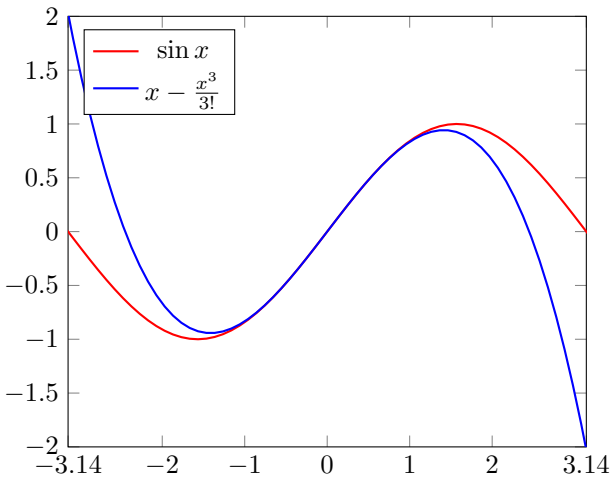
Taylor's Theorem claims that any function can be represented by a polynomial.

Why does this matter? Suppose we ask our computer to calculate $\sin\left(\frac{\pi}{3}\right)$. There is no sin operation in assembly, or sin unit in hardware. We can't just calculate it out of nowhere, but we *can* approximate it.

Here's a magic trick, let's try plotting $\sin x$ against $x^2 - \frac{x^3}{3!}$ and $x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$ for the heck of it.



We can see that they are pretty much almost the same. What about this:



So, we can see that the longer the polynomial goes on, the more accurate it becomes:

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} + \dots$$

That's cool, but, how the heck are we going to determine the polynomial? We will claim that we have a polynomial

$$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$$

And we would like to find all the coefficients.

We can first find a_0 by plugging in $x = 0$.

$$\begin{aligned} f(0) &= a_0 + a_1(0) + a_2(0)^2 + a_3(0)^3 \\ f(0) &= a_0 \\ \sin(0) &= 0 \end{aligned}$$

However, if we want to reuse the same trick to solve for a_1, it's not going to work. What do we do? *Take the derivative.* Taking the derivative will get rid of the first coefficient that we already found and remove x from the next one.

$$\begin{aligned} f'(x) &= a_1 + 2a_2 x + 3a_3 x^2 + 4a_4 x^3 + \dots \\ f'(0) &= a_1 \\ \cos(0) &= 1 \end{aligned}$$

We can then take the second derivative to find a_2:

$$\begin{aligned} f''(x) &= 2a_2 + 2 \cdot 3a_3 x + 4 \cdot 3a_4 x^2 + \dots \\ f''(0) &= 2a_2 \\ a_2 &= \frac{f''(0)}{2} \end{aligned}$$

And the third derivative to find a_3:

$$\begin{aligned} f^{(3)}(x) &= 3 \cdot 2a_3 + 4 \cdot 3 \cdot 2a_4 x + \dots \\ f^{(3)}(x) &= 3 \cdot 2a_3 \\ a_3 &= \frac{f^{(3)}(0)}{3!} \end{aligned}$$

We can start to see a pattern

$$\begin{aligned} f(x) &= \sum_{n=0}^{\infty} a_n x^n \\ &= \sum_{i=0}^{\infty} \frac{f^{(i)}(0)}{i!} \cdot x^i \end{aligned}$$

This may not look like much, but the function on the left could be anything, while the equation on the right can be easily calculated by computers.

For example, if $f(x) = e^x$

$$\begin{aligned} f(x) &= f(0) + f'(0) \cdot x + \frac{f''(0)}{2!} \cdot x^2 + \frac{f^{(3)}(0)}{3!} \cdot x^3 + \dots \\ &= 1 + 1 \cdot x + \frac{1}{2} \cdot x^2 + \frac{1}{3!} \cdot x^3 + \dots \end{aligned}$$

We also notice another important thing – the magnitude of each term depends largely on the power of x. Therefore, if x is small, it will become increasingly smaller as we go on. Suppose that x was around 0.1

$$\begin{aligned} f(x) &= a_0 + a_1 x + a_2 x^2 + a_3 x^3 \\ &= a_0 + a_1 \cdot 10^{-1} + a_2 \cdot 10^{-2} + a_3 \cdot 10^{-3} \end{aligned}$$

This is a very important property, since essentially, each term is a correction to the previous one. It implies that eventually, the terms will become so small that their corrections are negligible. There will of course, be some inaccuracies (which can be quantified). However, it is still very accurate.

On the other hand, if the terms are getting larger and larger, "you're kinda screwed". Anyway, we can approximate the polynomial with an error term:

$$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \frac{f^{(4)}(\xi)}{4!} \cdot x^4 \, \exists \, \xi \in [0, x]$$

The final term is our uncertainty/error term. Suppose we wanted to calculate $e^{0.1}$, we know that

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{f^{(4)}(\xi)}{4!} \cdot x^4 \, \exists \, \xi \in [0, x] \\ e^{0.1} &= 1 + x + \frac{0.1^2}{2} + \frac{0.1^3}{3!} + \frac{f^{(4)}(\xi)}{4!} \cdot 0.1^4 \, \exists \, \xi \in [0, 0.1] \end{aligned}$$

Because $e^{0.1}$ gives us the maximum, we use that.

$$\begin{aligned} e^{0.1} &= 1 + 0.1 + \frac{0.1^2}{2} + \frac{0.1^3}{3!} + \frac{f^{(4)}(0.1)}{4!} \cdot 0.1^4 \\ e^{0.1} &= 1 + 0.1 + \frac{0.1^2}{2} + \frac{0.1^3}{3!} + \frac{e^{0.1}}{4!} \cdot 0.1^4 \\ e^{0.1} &= 1.10516666... + \frac{e^{0.1} \cdot 0.1^4}{4!} \end{aligned}$$

So $e^{0.1} \in 1.1056666..., 1.1063455$.

In conclusion, every single function can be approximated using a polynomial. This is the basis for a lot of powerful algorithms.

Even Newton's method that we learned last class is essentially a Taylor series. We can find the error of Newton's method as well. Suppose that x_r is the real root, x_n is our guess and Δx (or $x_r - x_n$) is the distance from the root.

$$\begin{aligned} f(x_r) &= 0 \\ f(x_r) &= f(x_n + \Delta x) \\ &= f(x_n) + f'(x_n)(x_r - x_n) + \frac{f''(\xi)}{2} \cdot (x_r - x_n)^2 \end{aligned}$$

To figure out how much the error $E_n = x_r - x_n$ improves when we go from x_n to x_{n+1}:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \rightarrow f'(x_n)(x_n - x_{n+1}) = f(x_n)$$

If we plug $f(x_n)$ into the equation above:

$$\begin{aligned} 0 &= f'(x_n)(x_n - x_{n+1}) + f'(x_n)(x_r - x_n) + \frac{f''(\xi)}{2}(x_r - x_n)^2 \\ 0 &= f'(x_n)(x_r - x_{n+1}) + \frac{f''(\xi)}{2}(x_r - x_n)^2 \end{aligned}$$

This is important because $x_r - x_{n+1}$ is E_{n+1} and $(x_r - x_n)^2 = E_n^2$, therefore

$$E_{n+1} \approx E_n^2$$

In English, the uncertainty of the next iteration is the uncertainty of the current iteration squared (also known as *quadratic convergence* – one of Ajarn Piti's words to use in an interview).

- For the Bisection method, the error of the next term is the error of the previous, divided by two, which is much slower.
- However, the Bisection method guarantees that you will receive an answer, while Newton's method does not.

Numerical Derivatives

To us, something like $\frac{\mathrm{d}}{\mathrm{d}x}x^2 = 2x$ is trivial. A computer on the other hand, has no idea how to solve this. They can, however, calculate *numeric derivatives*. Let's look at the alternate definition for derivatives:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Can we calculate this with a computer? *No*, because of the limit. If we plug in zero, our computer's not going to be happy. However, we can plug in some super tiny value known as the *forward difference*.

$$f'(x) \approx \frac{f(x+0.01) - f(x)}{0.01}$$

From Taylor Series, we know that $f(x+h) = f(x) + hf'(x) + \frac{f''(\xi)}{2} \cdot h^2$, which can be manipulated to

$$f'(x) = \frac{f(x+h) - f(x)}{h} + \frac{f''(\xi)}{2} \cdot h$$
$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h)$$

Alternatively:

$$f(x+h) = f(x) + hf'(x) + \frac{f''(x)}{2} \cdot h^2 + \frac{f'''(\xi)}{3!} \cdot h^3$$
$$f(x-h) = f(x) - hf'(x) + \frac{f''(x)}{2} \cdot h^2 - \frac{f'''(\xi)}{3!} \cdot h^3$$

If we subtract the second formula from the first, we get

$$f(x+h) - f(x-h) = 2hf'(x) + O(h^3)$$
$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

Which is better because the error term is on the order of h^2, so the smaller h gets, the smaller the uncertainty becomes. This is known as *central difference*.

There is also another method that is even better known as the *five point stencil*.

In code, suppose we wanted to calculate the numerical derivative of $f'(0.5)$, where $f(x) = x^3$:

```python
def f(x):
    return x**3

def forward_difference(f, x, h):
    return (f(x+h) - f(x))/h

def central_difference(f, x, h):
    return (f(x+h) - f(x-h))/(2*h)

# Not so accurate
forward_difference(f, 0.5, 0.1)

# More accurate
forward_difference(f, 0.5, 10e-4)
central_difference(f, 0.5, 0.1)
```

Second derivative

We can calculate the second derivative by taking the first derivative of the first derivative:

$$f''(x) = \frac{f'(x+h) - f'(x-h)}{2h}$$

Calculating each of these terms separately, we get:

$$f'(x+h) = \frac{f(x+h+h) - f(x+h-h)}{2h} = \frac{f(x+2h) - f(x)}{2h}$$
$$f'(x-h) = \frac{f(x-h+h) - f(x-h-h)}{2h} = \frac{f(x) - f(x-2h)}{2h}$$

So in total:

$$f''(x) = \frac{\frac{f(x+2h)-f(x)}{2h} - \frac{f(x)-f(x-2h)}{2h}}{2h}$$
$$f''(x) = \frac{f(x+2h) - 2f(x) + f(x-2h)}{4h^2} + O(h^2)$$

Edge Detection

A lot of numerical algorithms are used with images. `matplotlib` provides us with some libraries to work with images as well. One cool thing we can do is to identify edges. But how?

Figure 2.1: Mario

We can identify edges by looking at the difference in color between a current pixel and its neighbors. For example, notice the edge between Mario's cap and his skin.

We must also ask ourselves how we are going to identify differences in color as well.

- We could quantify a range in which certain colors are the same or different, but what would that range be?

- We could take the averages of the color differences, but run into the issue of cancelling them out.

But, we could square the differences to get rid of the negatives and prevent them from cancelling out. If we have two $RGBA$ values, we could find the difference using:

$$diff = (r - r')^2 + (g - g')^2 + (b - b')^2 + (a - a')^2$$

The edge would be the pixel with the maximum color average among all neighbors.

3 | Interpolation and Integration

Interpolation

Suppose that we have four data points and would like to 'guess' the points in between them. We could just calculate the slope between each point and draw lines:

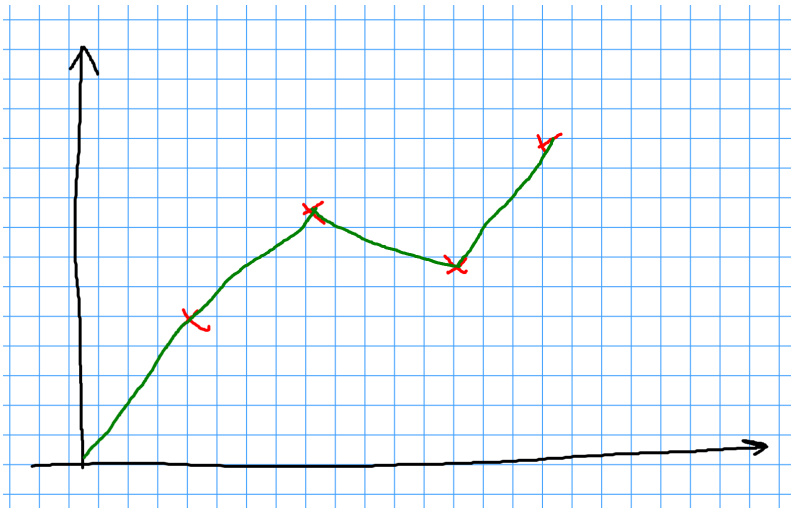


Figure 3.1: A bad guess

If we have two points, this'll always work. If we have three, we will require a quadratic / polynomial. If we have four points, that'll require a cubic function / polynomial of degree 3.

In principle, if we have n points, we will require a polynomial of degree $n-1$ in order to pass through all n points perfectly. We would like to determine this polynomial, known as the **Lagrange Interpolation Polynomial** (not the official name but "close enough"?)

The foundation

Suppose that we have four points: x_1, x_2, x_3 and x_4. We know that we require a polynomial of degree three to pass through all of them. Let's also suppose would like the polynomial to be 0 at x_2, x_3 and x_4, but *anything but zero* at x_1. Such a polynomial might look like

$$y = (x - x_2)(x - x_3)(x - x_4)$$

So that if we were to plug in x_1:

$$y(x_1) = (x_1 - x_2)(x_1 - x_3)(x_1 - x_4)$$

Ramping things up

Let's then suppose we have a more picky polynomial – maintaining the properties that the previous one had except we know require that the polynomial is exactly 1 at x_1.

How do we go about doing this? Well, if we knew that $y_{original}(x = x_1) = 5.2$, we could fix it by doing this:

$$y_{fixed} = \frac{y_{original}(x)}{5.2}$$

Or in a more generalized way:

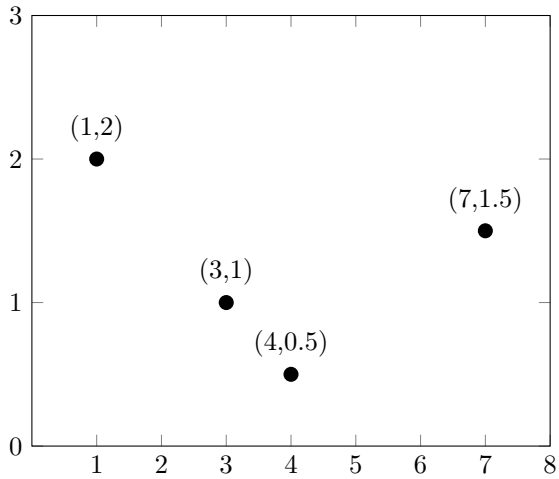
$$y = \frac{(x - x_2)(x - x_3)(x - x_4)}{(x_1 - x_2)(x_1 - x_3)(x_1 - x_4)} = W_1(x)$$

$W_1(x)$ is a polynomial that is 0 at all other x_i, but exactly 1 at x_1. We can construct the same manner of polynomial for other values as well. Why is this useful? What if we would like the function to be 3.14 at x_1? *Just multiply* W_1 *by* 3.14.

Applying $W_j(x)$

We know that we can construct a polynomial W_j which is 0 at all $x_i, i \neq j$ and 1 at x_j. What can we do with this?

Back to our original goal, say that we have four data points and we would like to find a polynomial that passes through every point.



We want a polynomial that has the following properties:

1. 2 at x_1
2. 1 at x_2
3. 0.5 at x_3

To satisfy all constraints, we can construct a polynomial using the $W_j(x)$ we discovered earlier

$$f(x) = 2W_1(x) + 1W_2(x) + 0.5W_3(x) + 1.5W_4(x)$$

Where

$$
\begin{aligned}
W_1(x) &= \frac{(x-x_2)(x-x_3)(x-x_4)}{(x_1-x_2)(x_1-x_3)(x_1-x_4))} \\
W_2(x) &= \frac{(x-x_1)(x-x_3)(x-x_4)}{(x_2-x_1)(x_2-x_3)(x_2-x_4)} \\
W_3(x) &= \frac{(x-x_1)(x-x_2)(x-x_4)}{(x_3-x_1)(x_3-x_2)(x_3-x_4)} \\
W_4(x) &= \frac{(x-x_1)(x-x_2)(x-x_3)}{(x_4-x_1)(x_4-x_2)(x_4-x_3)}
\end{aligned}
$$

Or in this case:

$$
\begin{aligned}
W_1(x) &= \frac{(x-3)(x-4)(x-7)}{(1-3)(1-4)(1-7))} \\
W_2(x) &= \frac{(x-1)(x-4)(x-7)}{(3-1)(3-4)(3-7)} \\
W_3(x) &= \frac{(x-1)(x-3)(x-7)}{(4-1)(4-3)(4-7)} \\
W_4(x) &= \frac{(x-1)(x-3)(x-4)}{(7-1)(7-3)(7-4)}
\end{aligned}
$$

Coding this up, we get:

```python
xs = np.array([2, 3, 4, 5])
ys = np.array([3, 1, 2, 2])

plt.plot(xs, ys, 'o')
plt.grid()

# Higher order functions (Functor)
# f takes in a float and returns a function g
# which takes in a float and returns a float
def w_factory(xs, i):
    denominator = np.prod([xs[i] - a for j, a in enumerate(xs) if i != j])
    def w(x):
        numerator = np.prod([x - a for j, a in enumerate(xs) if i != j])
        return numerator / denominator
    return w

w0 = w_factory(xs, 0) # -> polynomial that is 0 everywhere except at x[0]
print(w0(2), w0(3), w0(4), w0(5))

# Returns polynomial that passes through every xs, ys
def lagrange_factory(xs, ys):
    def lagrange(x):
        ws = [w_factory(xs, i) for i, _ in enumerate(xs)]
        return sum(y * ws[i](x) for i, y in enumerate(ys))

    return lagrange

lg = lagrange_factory(xs, ys)
```

```python

xxs = np.linspace(1, 6, 100)
plt.plot(xs, ys, 'o')
plt.plot(xxs, [lg(x) for x in xxs])
plt.grid()
```

When things can go wrong

We will have to note, however, that a Lagrange polynomial is not always accurate. It's merely taking a guess based on the points you feed into it. If your function is smooth enough, it can pretty much reconstruct the entire thing, but if we have something crazy, it might not give the results you expect.

- For example, using Lagrange interpolation on points for $\sin x$ from 0 to π? Fine. But, trying it on something like $\sin 200x$, maybe not so fine.

Numerical Integration

We all know how to integrate (hopefully), but suppose that Ajarn Chinda gives us this problem on our Calculus II final:

$$\int_1^{2.5} e^x \sin\left(x^2\right)\, dx$$

It's pretty much a death sentence. We may not be able to solve this by hand but we can, however, get a computer to evaluate the expression. We know that integration is the anti-derivative, the area under a curve.

When we performing numerical integration, we have a lower bound x_L and upper bound x_R ($\int_{x_L}^{x_R} f(x)\, dx$). We can break up this area under the graph between x_L and x_R into *trapezoids*.

- Each trapezoid has a width of $x_{i+1} - x_i$ (or Δx), a left-side height of $f(x_i)$ and a right side height of $f(x_{i+1})$

So, using the formula for area of trapezoids, the area of each trapezoid would be

$$\frac{1}{2} \cdot \Delta x \left(f(x_i) + f(x_{i+1})\right)$$

When we sum all n trapezoids up:

$$\begin{aligned}
Area &= \frac{1}{2}\Delta x \left(f(x_0) + f(x_1)\right) + \frac{1}{2}\Delta x \left(f(x_1) + f(x_2)\right) + \ldots + \frac{1}{2}\Delta x \left(f(x_{n-1}) + f(x_n)\right) \\
&= \frac{1}{2}\Delta x \left(f(x_0) + f(x_1) + f(x_1) + f(x_2) + f(x_2) + \ldots + f(x_n)\right) \\
&= \frac{1}{2}\Delta x \left[f(x_0) + 2 \cdot \sum_{i=1}^{n-1} f(x_i) + f(x_n)\right]
\end{aligned}$$

This is a good approximation, but... how accurate is it?

We would expect that

$$\begin{aligned}
\int_0^h f(x)dx &= \int_{x=0}^{x=h} f(0) + f'(0)x\, dx + \int_{h=0}^{h=h} O(h^2)\, dh \\
\int_0^h f\, dx &= [\text{trapezoid thing}] + O(h^3)
\end{aligned}$$

Which would imply that if we double the number of pieces h, our uncertainty would go down by a factor of h^3. However, this is not the case. When h is smaller, we need to add up more pieces. Skipping all the rigorous proofs, since we are adding all of them up, we need to multiply by n. Therefore, the true order of our uncertainty is $O(h^3 \cdot n)$, which becomes $O(\frac{1}{n^2})$. To conclude

$$\int_{x_L}^{x_R} f(x)\,dx = trapezoid + O\left(\frac{1}{n^2}\right)$$

You can actually reduce this to $O(\frac{1}{n^3}$ using another crazy algorithm, but the trapezoid method is pretty much the go-to algorithm.

The code is pretty simple:

```
def trapz(f, l, r, n):
    dx = (r - l) / n
    midsum = 2 * sum([f(l + i * dx) for i in range(1, n)])
    return 0.5 * dx * (f(l) + midsum + f(r))
```

In real life, people use `quad` from `scipy.integrate`

4 | Solving Matrices and Linear Regression

Suppose we have some equations

$$\begin{cases} x + y + z & = 6 \\ 2x + 4y + 3z & = 19 \\ -x + y - 2z & = -5 \end{cases}$$

We know we can solve this using linear algebra by representing the system of equations using matrices and row reducing

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 3 \\ -1 & 1 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 19 \\ -5 \end{bmatrix}$$

But is it possible to get a computer to do it for us? Let's examine the process in-depth. It works something like

1. We choose a pivot row p – the one we use to 'kill everyone'
2. $R_j \to R_j - \frac{R_{j,p}}{R_{p,p}} \times R_p$ for all rows besides p (where j is the row and p is the column)
3. For the right-hand side, $B_j \to B_j - \frac{R_{j,p}}{R_{p,p}} \times B_p$

For example, for the first column

- $R_2 \to R_2 - 2 \times R_1$
- $R_3 \to R_3 + 1 \times R_1$

Which gives us

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 2 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \\ 1 \end{bmatrix}$$

We then perform the same operation but with the second row as our pivot row

- $R_3 \to R_3 - R_2$

Which gives us

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \\ -6 \end{bmatrix}$$

The matrix we have now is an upper triangular matrix, which we have acquired through a process known as *forward elimination*. The code for this is as follows

```python
# Runtime of this is O(n^3) -- best for general case
def forward_elim(A, B):
    # Better to copy because the caller doesn't
    # know if things will be modified
    L = A.copy()
    R = B.copy()

    n_row, n_col = L.shape
    for p in range(n_row):
        for j in range(p + 1, n_row):
            co = L[j, p] / L[p, p]
            L[j] = L[j] - co * L[p]
            R[j] = R[j] - co * R[p]

    return L, R # upper triangular matrix, RHS

A = np.array([
    [1., 1, 1],
    [2., 4, 3],
    [-1, 1, -2]
])
B = np.array([
    6., 19, -5
])

L, R = forward_elim(A, B)
print(L)
print(R)
```

Now, we still need to reduce the upper half. We currently have

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \\ -6 \end{bmatrix}$$

We know we can just divide the lower-rightmost value by itself (and on the RHS as well):

$$ans_{last} = \frac{R_{last}}{L_{last}}$$

Or, in this case:

$$z = \frac{-6}{-2} = 3$$

But how do we solve for y? We know that that $2y + z = 7$, and since we have just solved for z, we can plug that bad boy in.

$$\begin{aligned} 2y + 1z &= 7 \\ 2y &= 7 - 1z \\ 2y &= 7 - 1(3) \\ y &= 2 \end{aligned}$$

Or in a more generalized way:

$$L_{j,j} \times Answer_j = R_j - \sum_{i=j+1}^{num\ cols} L_{j,i} \times Ans_i$$

$$Answer_j = \frac{1}{L_{j,j}} \left[R_j - \sum_{i=j+1}^{num\ cols} L_{j,i} \times Ans_i \right]$$

This is known as *backwards substitution*. We can code this up and combine it with forward elimination to solve for the answers:

```python
def forward_elim(A, B):
    L = A.copy()
    R = B.copy()

    n_row, n_col = L.shape
    for p in range(n_row):
        for j in range(p + 1, n_row):
            co = L[j, p] / L[p, p]
            L[j] = L[j] - co * L[p]
            R[j] = R[j] - co * R[p]

    return L, R

def backward_elim(L, R):
    n, _ = L.shape
    answer = np.zeros(n)

    # For the last one we can just divide
    answer[-1] = R[-1] / L[-1, -1]

    for j in range(n - 1, -1, -1):
        answer[j] = (1 / L[j, j]) * (R[j] - sum([L[j, i] * answer[i] \
        for i in range(j + 1, n)]) )

    return answer

def solve_matrix(A, B):
        L, R = solve_matrix(A, B)
        return backward_elim(L, R)

A = np.array([
    [1., 1, 1],
    [2., 4, 3],
    [-1, 1, -2]
])
B = np.array([
    6., 19, -5
])

L, R = forward_elim(A, B)
ans = backward_elim(L, R)
print(ans)
```

We can also just use `np.linalg.matrix.solve()`.

Linear Regression

If we are asked to plot a line through a set of points, we can probably distinguish between good and bad lines.

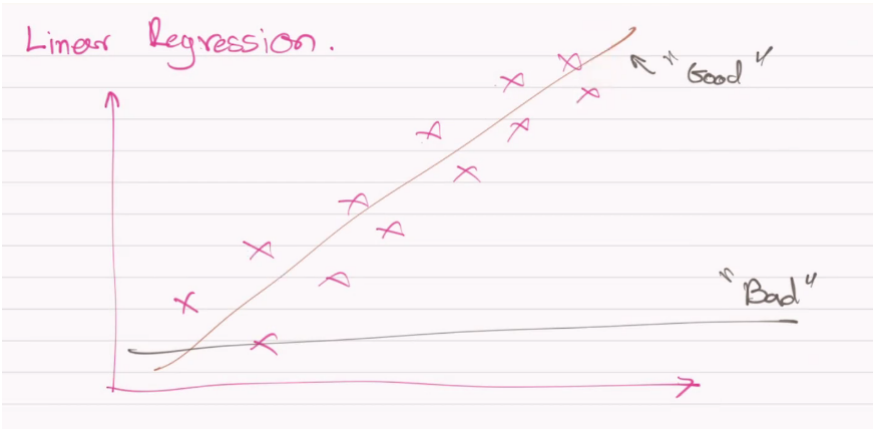


Figure 4.1: Examples of lines through points

Generally, good means there's less distance between the line and the points, but we're going to have to be a little more specific.

Suppose each point is a coordinate $\left(x^{(i)}, y^{(i)}\right)$. For the line, we know that there are two things that define it – the slope and intercept ($\hat{y} = mx + b$) – so if we can just find good values for both, we're set.

As for determining distance, the most natural way to do it would just be to measure the y distance between some x points on the line and your coordinate

$$distance = \hat{y}(x^{(i)}) - y^{(i)}$$

The closer the distance, the better. However, since each point will have a difference distance, we need to combine them somehow. We could take the average, but, if some of the distances come out to be negative, it's going to be a problem. We can fix that the way most issues with negative numbers are fixed, *by squaring it.*

Using all this, we can create a cost function that determines how good a line is:

$$cost(line) = \frac{1}{n}\sum_i (\hat{y}(x^{(i)}) - y^{(i)})^2$$
$$cost(m, c) = \frac{1}{n}\sum_i (mx^{(i)} + c - y^{(i)})^2$$

This is known as *least squares regression.* The lower the cost, the better (just like in real life, *nice*). So, our job is to find some values of m and c that minimize the cost. How do we do this? *Take the derivative*, or in this case, the partial derivative.

Generally, when handling functions of two variables, to find the critical points, we take the partial derivatives for both variables and set them equal to zero.

$$\frac{\partial}{\partial c} cost = 0, \frac{\partial}{\partial m} cost = 0$$

For the partial derivative with respect to c

$$\begin{aligned}\frac{\partial}{\partial c} cost = 0 &= \frac{1}{n}\sum_i \frac{\partial}{\partial c}(mx^{(i)} + c - y^{(i)})^2 \\ &= \sum_i 2(mx^{(i)} + c - y^{(i)}) \cdot \frac{\partial}{\partial c} c \\ &= m\sum_i x^{(i)} + nc - \sum_i y^i\end{aligned}$$

Where did the $\frac{1}{n}$ and 2 disappear to? Well, the other side of the equation is zero, so we can just eliminate them from the equation entirely. For the partial derivative with respect to m

$$\begin{aligned}\frac{\partial}{\partial m} cost = 0 &= \frac{1}{n}\sum_i \frac{\partial}{\partial m}(mx^{(i)} + c - y^{(i)})^2 \\ &= \sum_i (mx^{(i)} + c - y^{(i)})x^{(i)} \\ &= m\sum_i {x^{(i)}}^2 + c\sum_i x^{(i)} - \sum_i y^{(i)}x^{(i)}\end{aligned}$$

Now, you may be thinking, how the heck are we gonna solve for these? *How about we just use the algorithm we wrote to solve systems of equations above?*. We can feed it the array

$$\begin{bmatrix} \sum x^2 & \sum x \\ \sum x & n \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum yx \\ \sum y \end{bmatrix}$$

Which is known as the *Normal Equation*. The code looks something like

```python
def fit(xs, ys):
    # sum of (x^nx * y^ny)
    def s(nx, ny):
        return sum(xs**nx * ys**ny)

    n = len(xs)

    A = np.array([
        [s(2, 0), s(1, 0)],
        [s(1, 0), n]
    ])

    B = np.array([
        s(1, 1), s(0, 1)
    ])

m, c = fit(data_x, data_y) # from some data you have
plt.plot(data_x, data_y), '.')
plt.plot(data_x, [m * x + c for x in data_x])
```

Which should give us something that looks like:

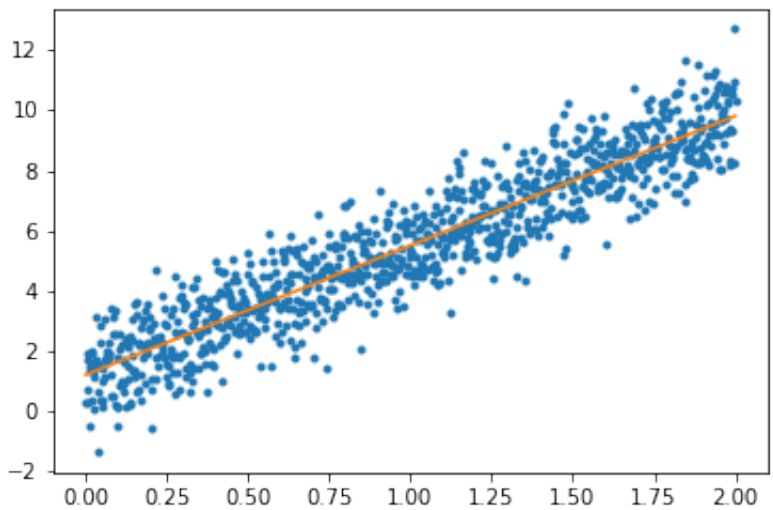


Figure 4.2: The result of our hard work

Going even further beyond

So, we have done linear regression, but what if we wanted to fit a line to a parabola? We know the formula for a parabola is

$$\hat{y} = ax^2 + bx + c$$

To find the best fit line, we want to find a good a, b and c. How do we find it? Do what we just did a second ago.

$$cost(a, b, c) = \sum_i (a{x^{(i)}}^2 + bx^{(i)} + c - y^{(i)})^2$$

Then, we take the partial derivative for a, b and c to get a 3×3 matrix.

$$\begin{aligned}
\frac{\partial}{\partial a} cost = 0 &= \sum_i (a{x^{(i)}}^2 + bx^{(i)} + c - y^{(i)})^2 \\
&= \sum_i (a{x^{(i)}}^2 + bx^{(i)} + c - y^{(i)}) \cdot x^2 \\
&= a \sum_i x^4 + b \sum_i x^3 + c \sum_i x^2 - \sum_i yx^2
\end{aligned}$$

$$\begin{aligned}
\frac{\partial}{\partial b} cost = 0 &= \sum_i (a{x^{(i)}}^2 + bx^{(i)} + c - y^{(i)})^2 \\
&= \sum_i (a{x^{(i)}}^2 + bx^{(i)} + c - y^{(i)}) \cdot x^2 \\
&= a \sum_i x^4 + b \sum_i x^3 + c \sum_i x^2 - \sum_i yx^2
\end{aligned}$$

We have three equations and three unknowns. We should be able to solve for it! If we do, we get

$$\begin{bmatrix} \sum x^4 & \sum x^3 & \sum x^3 \\ \sum x^3 & \sum x^2 & \sum x \\ \sum x^2 & \sum x & n \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} \sum yx^2 \\ \sum yx \\ \sum y \end{bmatrix}$$

Generally, equations will never look this nice. However, you can use log to help you out of a lot of tricky multiplication/exponent situations.

Higher Dimensions

We can go beyond a single variable as well. Suppose that we want to estimate the height of a child using the heights of their parents

$$\hat{h_c}(h_d, h_m) = c_m h_m + c_d h_d + h_0$$

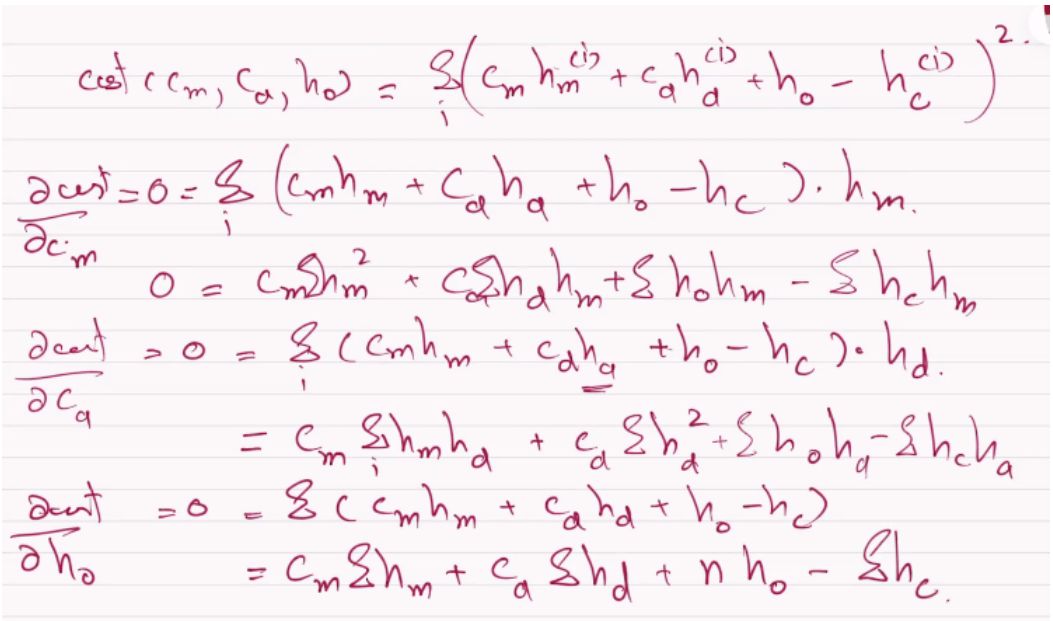
where c_m and c_d are modifiers and h_0 is the offset. If our function is accurate, then the cost would be low:

$$cost(c_m, c_d, h_0) = \sum_i (\hat{h}_c(h_d^{(i)}, h_m^{(i)}) - h_c^{(i)})^2$$

To find c_m, c_d, h_0, we take the partial derivative for each variable again:

$$cost(c_m, c_d, h_0) = \sum_i (c_m h_m^{(i)} + c_d h_d^{(i)} + h_0 - h_c^{(i)})^2$$

Which looks like:



$$cost(c_m, c_d, h_0) = \sum_i \left(c_m h_m^{(i)} + c_d h_d^{(i)} + h_0 - h_c^{(i)}\right)^2.$$

$$\frac{\partial cost}{\partial c_m} = 0 = \sum_i (c_m h_m + c_d h_d + h_0 - h_c) \cdot h_m.$$

$$0 = c_m \sum h_m^2 + c_d \sum h_d h_m + \sum h_0 h_m - \sum h_c h_m$$

$$\frac{\partial cost}{\partial c_d} = 0 = \sum_i (c_m h_m + c_d h_d + h_0 - h_c) \cdot h_d.$$

$$= c_m \sum_i h_m h_d + c_d \sum h_d^2 + \sum h_0 h_d - \sum h_c h_d$$

$$\frac{\partial cost}{\partial h_0} = 0 = \sum (c_m h_m + c_d h_d + h_0 - h_c)$$

$$= c_m \sum h_m + c_d \sum h_d + n h_0 - \sum h_c.$$

Figure 4.3: Taking the partial derivative with respect to each variable

I was going to typeset this but my wrists were like

Figure 4.4: Everything's a JoJo reference at this point

Anyway, each equation then becomes

$$\begin{bmatrix} \sum h_m^2 & \sum h_m h_d & \sum h_m h_0 \\ \sum h_m h_d & \sum h_d^2 & \sum h_d h_0 \\ \sum h_m & \sum h_d & \sum n \end{bmatrix} \begin{bmatrix} c_m \\ c_d \\ h_0 \end{bmatrix} = \begin{bmatrix} \sum h_c h_m \\ \sum h_c h_d \\ \sum h_c \end{bmatrix}$$

And there we have it! We have now ascended to higher dimensions.

5 | Optimization

Optimization is the process of finding x such that $f(x)$ is minimized. It is also known as $\underset{x}{argmin}\, f(x)$ Suppose that we wanted to find the minimum of

$$f(x) = (x-2)^2 + 3$$

. We can easily minimizing this by plugging in 2

$$f(2) = (2-2)^2 + 3 = 3$$

This looks and sounds a little mathy, but, as many people say: *every problem is an optimization problem.*

Root finding

Suppose that we wanted to solve

$$\sin(x) = x$$

Can we turn this into an optimization problem? Yeah.

$$\underset{x}{argmin}(sin(x) - x)^2$$

By squaring, the only place where x can be minimized is when the two are equal.

Linear regression

If we have a linear regression problem, we can also turn it into an optimization problem

$$\underset{m,c}{argmin} \sum (mx^i + c - y^i)^2$$

Gaussian elimination

We can even do Gaussian elimination!? If we have the matrices

$$[A]\,[\vec{x}] = \left[\vec{b}\right]$$

We can minimize

$$\underset{\vec{x}}{argmin} ||A\vec{x} - \vec{b}||^2$$

Even more

You can even solve what to eat for lunch based on yuminess and price, or figure out Algo homework with optimization!

Optimization in 1D

Let's return to our first equation:

$$f(x) = (x-2)^2 + 3$$

How can we solve this with a computer? We can try plotting it out and seeing where it goes, but that's going to be expensive, computationally. Alternatively, we could just choose a point on the graph and check its slope.

- If the slope of the point is negative, we know our next guess should be to the right of that point
- If the slope is positive, we know our next guess should be to the left of our guess

This is, of course, finding the local minima (finding global minima is very hard). Anyway, the equation looks something like

$$x_{n+1} = x_n + \left(\frac{-f'(x)}{|f'(x)|}\right)$$

However, this equation only gives us the direction. We also need a step size – how much to move by with each iteration. Let's try some experimenting.

If we start our guess at $x = 4$, have a step size of 1.1 and run 10 iterations, we get something that looks like

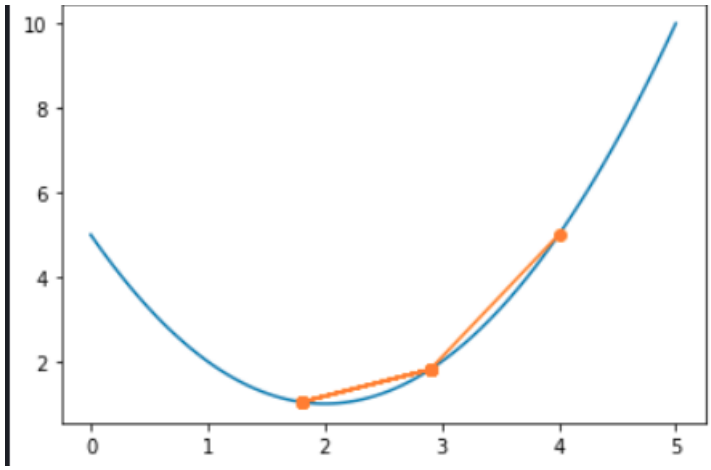


Figure 5.1: $x = 4$, $step = 1.1$, $n = 10$

If we reduce the step size to 0.01 and go for 100 iterations, we get this:

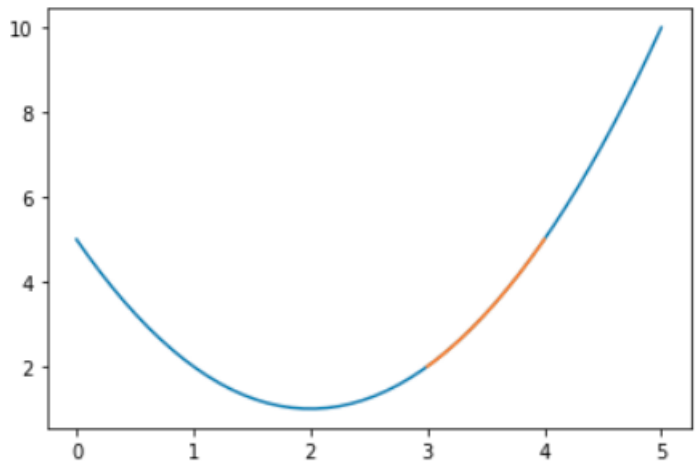


Figure 5.2: $x = 4$, $step = 0.01$, $n = 100$

If we increase the iterations by enough, we will notice the guesses will bound around/oscillate between the answer, so constant step size is not really optimal. So far we've learned that

- Big step size is fast, but inaccurate
- Small steps are slow, but accurate
- We want something that's accurate but fast

The solution is obvious! We can start big, while we are far away from the solution, and get smaller when we are closer to the solution. New problem: *how do we know if we are close or far to the solution?* Well, we know that the slope should be zero at the solution.

- If we are far, the slope should be pretty high
- If we are close, the slope should be pretty small

Therefore, step size should be proportional to the slope (or rather, the absolute value of the slope, since we can have big negatives):

$$step = \lambda \cdot |f'(x)|$$

Where λ is the **learning rate** – popular in machine learning.

- A large learning rate means to take a big step
- A small learning rate means to take a small step

We will have to choose an appropriate learning rate for each problem. If we have a super super large learning rate, it will scale with the slope and may become so large that it doesn't converge. If it does work, it'll be super fast, but it's not likely to. On the other hand, if our learning rate is super small, every step will be tiny and we will require many more iterations to reach our destination (if we even reach it at all). Putting this together with what we have so far:

$$\begin{aligned} x_{n+1} &= x_n - \frac{f'(x)}{|f'(x)|} \cdot \lambda \cdot |f'(x)| \\ &= x_n - \lambda f'(x) \end{aligned}$$

This algorithm is known as **Gradient Descent in 1D**. We can implement this in Python easily (in this case, for $f(x) = (x-2)^2 + 1$):

```python
def grad_desc_1d(fp, x_0, learning_rate, n):
    guess = x_0
    guesses = [guess]
    for i in range(n):
        guess = guess - learning_rate * fp(guess)
        guesses.append(guess)
    return guess, np.array(guesses)

def f(x):
    return (x-2)**2 + 1

def fp(x):
    return 2*(x-2)

x, guesses = grad_desc_1d(fp, x_0=4, learning_rate=0.1, n=100)
xs = np.linspace(0, 5, 100)
plt.plot(xs, f(xs))
plt.plot(guesses, f(guesses), 'o-')
```

Multi-Dimensional Gradient Descent

Suppose we have the function

$$f(x, y) = (x - 2)^2 - xy + y^2 + 1$$

We can plot this in 3D and get this cool looking visualization, but it really tells us nothing:

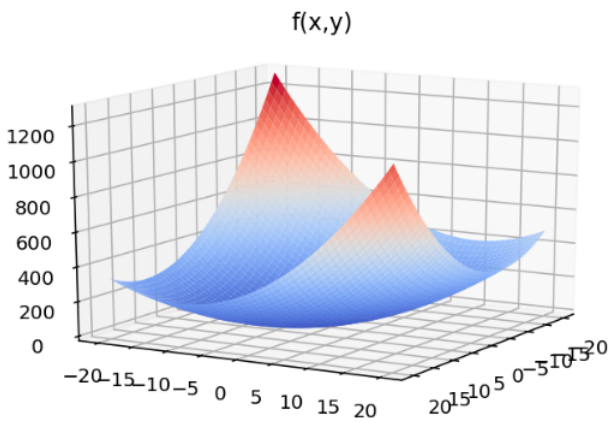


Figure 5.3: A 3D visualization of $f(x, y) = (x - 2)^2 - xy + y^2 + 1$

A better way would be to use a contour graph:

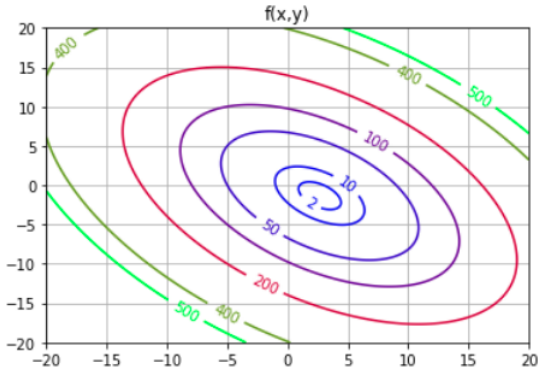


Figure 5.4: A contour graph of $f(x, y) = (x - 2)^2 - xy + y^2 + 1$

The minimum that we want is at the ring in the middle. Finding this minimum is analogous to *tossing a marble into a bowl and seeing where it rolls down to rest* (what an awesome analogy). So, in one-dimension, we did something like this

$$f(x_n + \Delta x) = f(x_n) + f'(x_n) \cdot \Delta x$$

Since we are finding the minimum, we hope that $f(x_n + \Delta x)$, or x_{n+1}, should be a lot smaller than $f(x_n)$. So, we would like $f'(x_n) \cdot \Delta x$ to be as small as possible.

- When $f'(x_n)$ is positive, we want Δx to be negative
- When $f'(x_n)$ is negative, we want Δx to be positive

So, if we have $f(x_n, y_n)$, we can perform Taylor Series Expansion on it:

$$f(x_n + \Delta x, y_n + \Delta y) = f(x_n, y_n) + ...$$

For the ..., we want to know how much the updated point will change if we move x alone, and how much it will change if we move y alone.

$$f(x_n + \Delta x, y_n + \Delta y) = f(x_n, y_n) + \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y + H.O$$

The $\frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y$ is actually a dot product:

$$\begin{aligned} f(x_n + \Delta x, y_n + \Delta y) &= f(x_n, y_n) + \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y \\ &= f(x_n, y_n) + (\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}) \cdot (\Delta x, \Delta y) + H.O. \end{aligned}$$

This $(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}) \cdot (\Delta x, \Delta y)$ is known as a **gradient function**. If you know your Cal3B (which I don't), it is equivalent to:

$$\vec{\nabla} f \cdot \Delta \vec{r}$$

Where the first term is the gradient and the second is the movement. We can rewrite our previous formula as:

$$f(\vec{r}_n + \Delta \vec{r}) = f(\vec{r}_n) + \vec{\nabla} f \cdot \Delta \vec{r} + H.O$$

So, the gradient of $\vec{\nabla} f$ is fixed, all we can choose is how to change $\Delta \vec{r}$. To get it to be as negative as possible, we will choose the vector opposite of the gradient. To visualize:

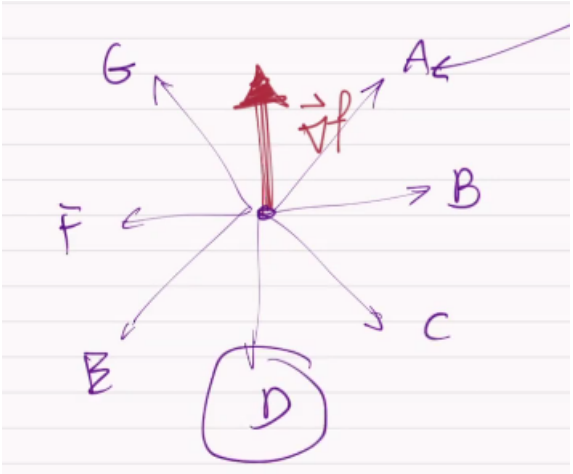


Figure 5.5: Gradient Visualization

So, our update rule looks something like

$$\begin{aligned}\vec{x}_{n+1} &= \vec{x}_n - \frac{\vec{\nabla} f(\vec{x}_n)}{|\vec{\nabla} f(\vec{x_n})|} \cdot \lambda \cdot |\vec{\nabla} f(\vec{x_n})| \\ &= \vec{x}_n - \lambda \vec{\nabla} f(\vec{x}_n)\end{aligned}$$

Let's try this on the function above.

```python
def grad(f, v, h=0.01):
    # df/dx --> (f(v+[h, 0]) - f(v))/h
    # df/dx --> (f(v+[0, h]) - f(v))/h

    n_var = len(v)
    ret = np.zeros(n_var)

    for i in range(n_var):
        dv = np.zeros(n_var)
        dv[i] = h
        diff = (f(v + dv) - f(v - dv))/2/h
        ret[i] = diff

    return ret

def f(v):
    x, y = v
    return (x - 2)**2 + x * y + y**2 + 1

def grad_desc(f, v0, learning_rate, n):
    guess = np.array(v0)
    guesses = [guess]
    for i in range(n):
        guess = guess - learning_rate * grad(f, guess)
        guesses.append(guess)
    return guess, guesses

v, vs = grad_desc(f, [10., -15.], 0.1, 40)

```

```
for x, y in vs:
    plt.plot(x, y, 'o')

# Plot the contour

from matplotlib import cm
#see http://matplotlib.org/examples/pylab_examples/contour_demo.html

nx = 200
ny = 200
x = np.linspace(-20,20,nx)
y = np.linspace(-20,20,ny)
xgrid, ygrid = np.meshgrid(x, y)
#print ygrid
uz = np.zeros((nx,ny))

for ix in range(nx):
    for iy in range(ny):
        xx = x[ix]
        yy = y[iy]
        uz[iy,ix] = f([xx,yy]) ##<<<<<<This is not a typo: [iy,ix]

ucs = plt.contour(x, y, uz, [2,10,50,100,200,400,500], cmap=cm.brg)
plt.clabel(ucs, inline=1, fontsize=10)
plt.grid()
plt.title('f(x,y)')
```

Which provides this beautiful visualization:

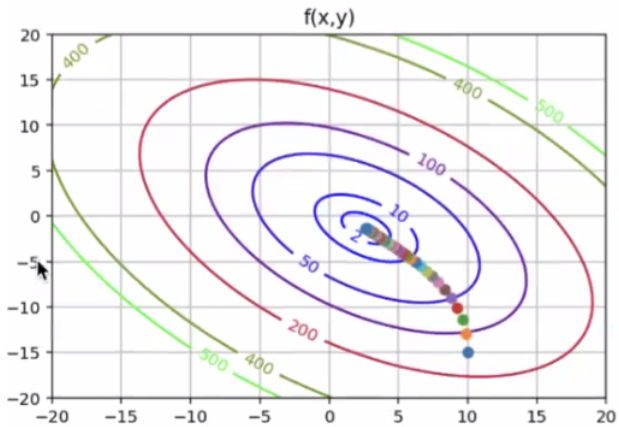


Figure 5.6: Gradient descent on $f(x, y) = (x - 2)^2 - xy + y^2 + 1$

Maximization

We learned about minimzation, but what about maximization? It's as simple as:

$$argmax f(x) = argmin - f(x)$$

Separating Data

Suppose we have this data:

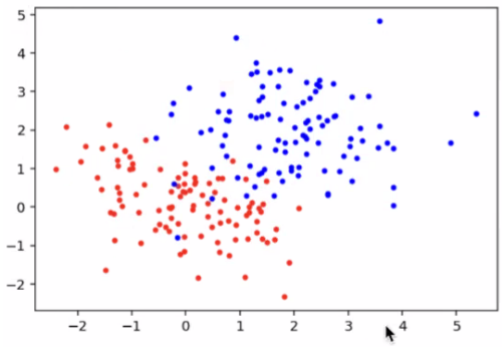


Figure 5.7: Red and blue data

How do we come up with a good line to separate it? We can see that there is a clear distinction, but it's just a matter of telling our computer how to compute it. We know that a good line has blue *above* the line and red *below* the line.

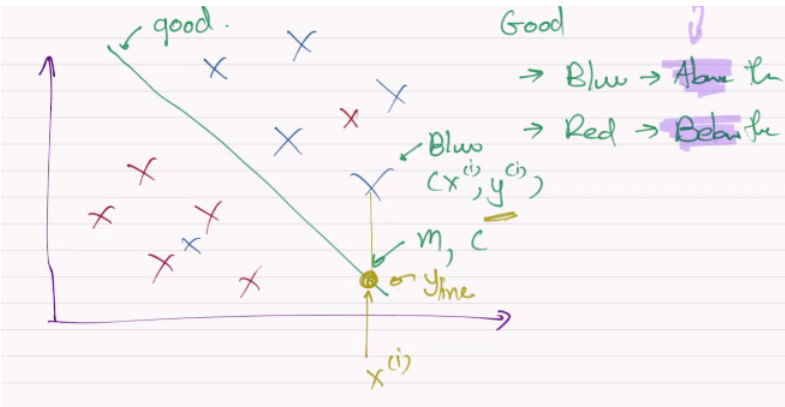


Figure 5.8: A good line

We can calculate the difference between y points on the line and the data

$$\Delta y = mx^{(i)} + c - y^{(i)}$$

Above the line means $\Delta y \leq 0$ and below means $\Delta y \geq 0$. We can give a penalty as follows: if a data point is red and above the line, or blue and below the line – penalize. But, we don't want to penalize all points by the same amount in case we have outliers, so we can penalize by Δy to the cost.

```
from scipy.optimize import minimize

np.random.seed(234)
n1 = 100
n2 = 100
mean = [0, 0]
cov = [[1, -0.5], [-0.5, 1]]
```

```python
r1, r2 = np.random.multivariate_normal(mean, cov, n1).T
#plt.plot(r1, r2, '.r')

mean = [2, 2]
cov = [[1, 0], [0, 1]]
b1, b2 = np.random.multivariate_normal(mean, cov, n2).T
#plt.plot(b1, b2, '.b')

c = np.zeros(n1+n2)
c[:n1] = 1.
c[n1:] = 0.
#print c

d = np.zeros((n1+n2,2))
d[:n1,0] = r1
d[:n1,1] = r2
d[n1:,0] = b1
d[n1:,1] = b2

# Minimize function
def f(v):
    m, b = v
    cost = 0
    for color, (x, y) in zip(c, d):
        blue = color == 0
        red = not blue
        dy = m * x + b - y
        above = dy < 0
        below = dy > 0
        if (red and above) or (blue and below):
            cost += abs(dy)
    return cost

res = minimize(f, [-1., 1.])

plt.plot(d[c==1,0], d[c==1,1],'.r')#note boolean indexing
plt.plot(d[c==0,0], d[c==0,1],'.b')#note boolean indexing
xs = np.linspace(-2, 5, 100)
m, b = res.x
plt.plot(xs, m*xs + b)
```

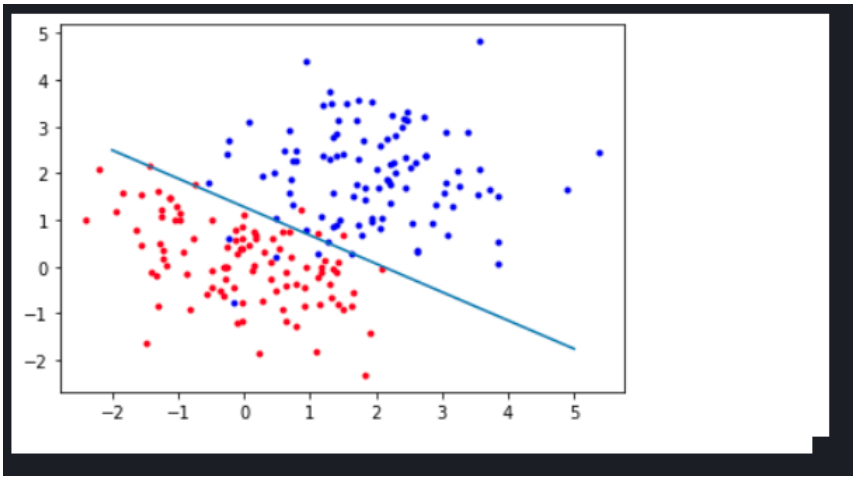


Figure 5.9: Separated data

Ajarn Piti's Game Recommendations

Ajarn Piti recommends

- Dyson Sphere Program
- Shapez.io
- Vampire Survivors

6 | Monte Carlo Simulation

Random Numbers

How do video games choose which characters to give you when you open a loot crate? *Random number generation.* But how exactly do computers generate random numbers? Computers don't really have a true concept of randomness – in general, they just operate based on our instructions. In computers, we have something known as a *Pseudo-Random Number Generator* (PRNG). It looks random, but it's not really because it is deterministic.

Linear Congruential Generator

One of the most popular PRNGs is called the Linear Congruential Generator. It works by taking a number and generating another number from it uses some set values:

$$x_{n+1} = ax_n + c \, mod \, m$$

However, that means we need some number to start with – x_0. This is known as the *seed.* Suppose that in this case:

- $x_0 = 3$, $a = 2$, $c = 2$, $m = 8$

There are actually some requirements that the values need to meet in order to be truly random but, go read those on Wikipedia. These values would give us something like:

$$\begin{aligned} x_0 &= 3 \\ x_1 &= 2 \times 3 + 2 \, mod \, 8 = 0 \\ x_2 &= 2 \times 0 + 2 \, mod \, 8 = 2 \\ x_3 &= 2 \times 2 + 2 \, mod \, 8 = 6 \\ &\vdots \end{aligned}$$

We can get Python to calculate this easily. However, choosing some values may lead to a cycle.

In Real Life

In real life, we have a couple of random number generators (which have linear congruential generators behind them)

- `np.random.random` – gives you a float between 0 and 1
- `np.random.randint` – gives you a random integer in some range
- `np.random.rand` – gives a random float for normal distribution with mean 0 and standard deviation 1.

You could also set a seed to get 'fixed' random values

```python
import numpy as np
import matplotlib

# Uniform random distribution
np.random.seed(26283)
numbers = np.random.random(10000)
plt.hist(numbers, bins=30)
```

Manipulating random number behavior

Given NumPy's random number generation stuff, how can we manipulate these random numbers? Let's look at some examples.

What if we wanted to generate a random float between 0 and 5?

```python
# Generates a float from 0 to 1 and multiplies by 5 to shift
np.random.random(10000) * 5
```

What if we wanted to generate a random float from -1 to 1?

```python
# Mult by 2 gives us from 0 to 2 so we subtract by 1 to shift
np.random.random(10000) * 2 - 1
```

What if we wanted to get a random number from a normal distribution with mean 2 and standard deviation 10?

```python
np.random.randn(10000) + 5
```

Truly random numbers

To create truly random numbers, we would need access to the physical world. There are a couple of ways to do this

- Network requests
- CPU temperature
- Letting a radioactive substance decay But that might be a bit overkill for just generating random numbers for our programs.

Monte Carlo Simulations

Suppose we wanted to get a random value for rolling a loaded die with the probabilities

Number	1	2	3	4	5	6
Probability	0.3	0.2	0.1	0.2	0.1	0.1

We could just do `np.round(np.random.random(1000) * 6 + 1)`, which would give us random numbers between 1 and 6, but the probabilities wouldn't be correct. We could instead write a function that would generate a random value for us based on those probabilities

```python
def gacha():
    n = np.random.random() # [0 - 1] float

    if n < 0.3:
        return 1
    elif n < 0.5:
        return 2
    elif n < 0.6:
        return 3
    elif n < 0.8:
        return 4
    elif n < 0.9:
        return 5
    else:
        return 6

numbers = [gacha() for i in range(10000)]
plt.hist(numbers, bins=20)
```

This is what is known as a Monte Carlo Simulation. Instead of analytically working out the answers of complex problems, we can just use probability and run a ton of trials to simulate.

We can also use it for things like expected value:

```python
# -- Expected value of dice result --
# Analytically
E_x = 0.3*1 + 0.2*2 + 0.1*3 + 0.2*4 + 0.1*5 + 0.1*6
print(E_x)

# Simulation
trials = [gacha() for _ in range(10000)]
np.mean(trials)

# -- Expected value of A+B where A and B are two loaded dice --
# Analytically
print(E_x + E_x) # Linearity of expectation

#Simulation
trials = [gacha() + gacha() for _ in range(10000)]
np.mean(trials)

# -- Expected value of AB --
# Analytically
print(E_x * E_x) # E[A * B] = E[A] * E[B]

# Simulation
trials = [gacha() * gacha() for _ in range(10000)]
np.mean(trials)
```

There will however, be situations that are much harder to determine the answer for analytically. For example, if we have two people roll two dice and then take the product of their results, what would be the probability of getting a draw? A nightmare to do analytically but for simulation:

```
result = [gacha() * gacha() == gacha() * gacha() for _ in range(10000)]
np.sum(result) / 10000 # since True is 1 and False is 0
```

We could also do something even harder – same as the problem before, but what is our probability of winning if we get three as our first die?

```
results = [3 * gacha() > gacha() * gacha() for _ in range(10000)]
np.sum(results) / 10000
```

As you can see, super useful. It can even be used to simulate the probability of winning at Blackjack given some conditions.

Monte Carlo Integration

Suppose we wanted to integrate something in multiple dimensions

$$\iiint_C x^2(y-1)^2(z-2)^2 \exp(xyz)\, dx\, dy\, dz$$

Over a cube $x \in [-1, 1], y \in [-1, 1], z \in [-1, 1]$. Physically, we would do:

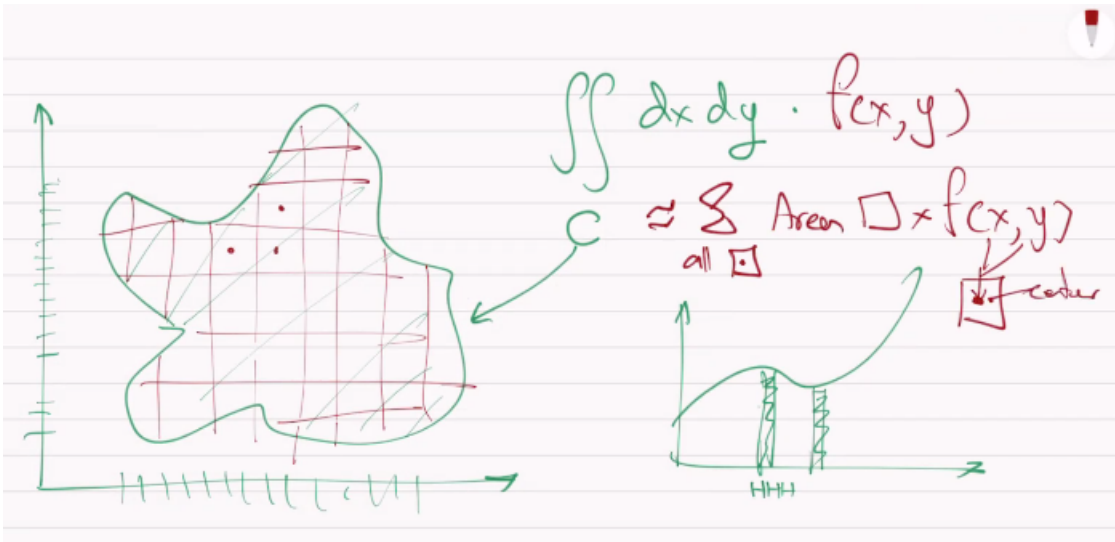


Figure 6.1: Integration in multiple dimensions

Basically, it involves subdividing the shape of the function into a ton of pieces, calculating the area for all those pieces and then summing them up. However, if we were to do this well, it would take, say for example, a thousand pieces. If we had a thousand x pieces and a thousand y pieces, summing all of them up would take 10^6 iterations. But, if we were to add another dimension, this would become 10^9 – a lot.

We can generalize this by saying that if we have some integral of dimension d, subdivided by n pieces, the runtime for this method would be $O(n^d)$. This has a super cool name: **The Curse of Dimensionality**. So, summing all the pieces is pretty much a terrible idea if $d \geq 3$. So what do we do?

We can try something called *Monte Carlo Integration* – summing only random pieces:

$$Area \approx \sum_{random} area \, \square \times f(x, y)$$

Let's start with this, suppose we wanted to integrate a rectangle:

$$\iiint_C dx \, dy \, f(x, y)$$

We could just keep choosing a random point in C and use it to get x and y, and just take the average for the area.

Abstractly, we basically want to randomly choose a thousand or so points, calculate the sum of the fixed total area of those points divided by n multiplied by $f(x, y)$:

$$\sum_n \frac{total\ area}{n} \times f(x, y)$$

To make it more concrete:

```python
# random a point in big cube, throw it away if it's not inside the circle

def get_point():
    x, y, z = np.random.random(3) * 2 - 1 # from -1 to 1
    return x, y, z

s = 0
n_point = 0
n = 100000

for _ in range(n):
    x, y, z = get_point()
    if x**2 + y**2 + z**2 < 1: # if it is in a circle
        s += x**2 * (y-1)**2 * (z-2)**2 * np.exp(x*y*z)
        n_point += 1
    else:
        s += 0 # not in the circle

print(s*8/n) # 8 is the area of the cube???
```

As we know from earlier, our final answer is not nearly as important as the error. But, how do we determine how good this algorithm really is if it's random?

- A good measure is the width of the distribution.

The answer for anyone using this algorithm is

$$Answer = \frac{total\ area}{n} \sum_{n\ points} f(x, y)$$

Where *Answer* is a random variable. Finding the standard deviation is kind of tough, but we

can easily find the variance:

$$
\begin{aligned}
Var[Answer] &= Var\left[\frac{total\ area}{n}\sum_{n\ points} f(x,y)\right] \\
&= (\frac{total\ area}{n})^2 Var \times \left[\sum_{n\ points} f(x,y)\right] \\
&= (\frac{total\ area}{n})^2 \times n \times Var\left[f(x,y)\right] \\
&= \frac{1}{n} \times \square
\end{aligned}
$$

Here comes the main point – the variance of our answer is $\frac{1}{n}$ multiplied by some constant, which means our standard deviation is on the order of $\frac{1}{\sqrt{n}}$.

> "This is a very important fact, remember this for the rest of your life."
>
> – Piti Ongmongkolkul, 2022

Why is it so important? *It means this method sucks.* If we increase the factor of our n by 100 from 1000 to $100,000$, the width of our distribution is only 10 times smaller (which is sublinear). So, pretty sad.

Geometric Brownian Motion

One popular way to simulate stock prices is to use Geometric Brownian Motion. Suppose that a stock's price at $t + dt$ is related to the stock price at time t by

$$s(t + dt) = s(t) + s(t)(\mu dt + \epsilon\sigma\sqrt{dt})$$

where

1. μ is the mean rate of return. Typically % per year.
2. ϵ is a normally distributed random variable with mean 0 and variance of 1.
3. σ is volatility. It is typically quoted as % per $\sqrt{\text{year}}$ and represents how wildly a stock can fluctuate.

We can do some cool stuff with this.

Suppose that the volatility(σ) is 28% per $\sqrt{\text{year}}$ and mean rate of return is 8.77%. The stock today is 119.21. To generate the price for tomorrow given the price today:

```
def next_stock(s, mu, sigma, dt):
    return s + s * (mu * dt + np.random.randn() * sigma * np.sqrt(dt))
```

To generate all the prices for one trading year

```
# 252 days is one trading year
def gen_stock(s, mu, sigma, dt=1/252, n=252):
    ret = [s]
    for i in range(n):
        s = next_stock(s, mu,  sigma, dt)
```

```
        ret.append(s)
    return ret
```

To get the expected value of the stock at the end of the year:

```
stocks = [gen_stock(119.21, mu=0.0877, sigma=0.28) for _ in range(1000)]

for stock in stocks:
    plt.plot(stock)

np.mean([s[-1] for s in stocks]) # Looks like a good investment
```

To determine the probability that a stock will dip below 70 at any time of the year:

```
dip = [np.any(np.array(s) < 70) for s in stocks]
print(np.sum(dip) / len(dip)) # there's about a four percent change we're going to lose a to
```

We can get even crazier and determine the probability that the stock will dip below 100 and come back to 110 within a year

```
def dip_and_back(stock):
    below_hundred = False
    for v in stock:
        if v < 100:
            below_hundred = True
        if below_hundred and v > 110:
            return True
    return False

dip = [dip_and_back(s) for s in stocks]
print(np.sum(dip) / len(dip))
```

And finally, most importantly, we can determine the probability of making a profit from selling our stocks:

```
def meets_sell_requirements(stock):
    original = stock[0]
    for v in stock:
        if (v > original + 10) or (v < original - 5):
            return True
    return False

def profit_made(stock):
    original = stock[0]
    for v in stock:
        if v > original + 10:
            return Truef
    return False

sold = [meets_sell_requirements(s) for s in stocks]
profit = [profit_made(s) for s in stocks]

print(np.sum(sold) / len(sold))
```

```
print(np.sum(profit) / len(profit))
```

7 | Hill Climbing

In this section we will look at Hill Climbing / Random Search / Simulated Annealing – an algorithm that sounds super dumb but is really useful.

So far we have learned about gradient descent and optimization, which is super useful, however, we can't always calculate a gradient for everything. For example, let us consider the problem of finding a triangle with the maximum are from 500 random data points:

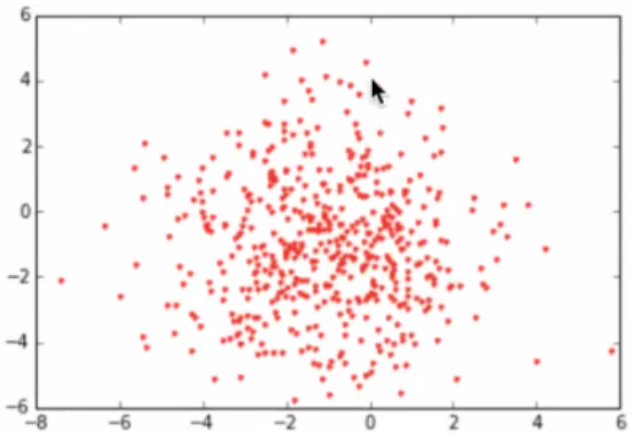


Figure 7.1: 500 scattered data points

How might we approach this?

- We could try all possible combinations of three points, but that would take 500^3 iterations, even if it guarantees the best possible answer. Plus, we're pretty much screwed if we have 10,000 points or someting like that.

We can instead do something that sounds really dumb: change the points randomly.

- If the change increases the area, that means we're getting closer
- If the change decreases the area, that means it's a bad idea

This algorithm surpasses greedy to *drunk and greedy* status.

The code for it works something like this

- We have a cost function that 'scores' some variables of interest (in this case the area of a triangle with points p_1, p_2, p_3. The current set of points are referred to as the *state*.
- We have a function that *perturbs* the state – changes one value randomly.

- After perturbing, we compare costs and if the cost is better, swap out the best state so far with our new state
- Do that for 20K iterations or something (seems like a lot but much better than 500^3)

(Had to use Ajarn Piti's code for this since I deleted my Jupyter Notebook file for some reason...)

```
np.random.seed(13)
n = 500
x = np.random.randn(n)*2-1
y = np.random.randn(n)*2-1
plt.plot(x,y,'.r')

def triarea(x1,y1, x2,y2, x3,y3):
    #shoelace formula
    return 0.5*abs(x1*y2 + x2*y3 + x3*y1 - y1*x2 - y2*x3 - y3*x1)

def triarea2(r):
    x1,y1 = r[0]
    x2,y2 = r[1]
    x3,y3 = r[2]
    return triarea(x1,y1, x2,y2, x3,y3)

def cost(state):
    return triarea2(state)

def random_state(): # get initial random state
    n_points = len(x)
    return [[x[i], y[i]]for i in np.random.randint(n_points, size=3)]

def perturb(old_state):
    n_points = len(x)

    # Need to copy so old state doesn't get changed
    ret = [s for s in old_state] # shallow copy
    point_to_change = np.random.randint(3)

    change_to_index = np.random.randint(n_points)
    ret[point_to_change] = [x[change_to_index], y[change_to_index]]
    return ret


def hill_climbing(initial_state, n):
    current_best = initial_state
    current_score = cost(initial_state)

    for i in range(n):
        to_try = perturb(current_best)
        to_try_score = cost(to_try)
        if to_try_score > current_score:
            current_best = to_try
            current_score = to_try_score
            print(i, current_best)
    return current_best

```

```
guess = hill_climbing(random_state(), 10000)
plt.plot(x,y,'.r')
for i in range(3):
    plt.plot(guess[i][0], guess[i][1], '^b')
```

Scheduling

Suppose that MUIC has 200 classes with 200 finals. If Nawat is taking two classes and they both have the same final exam time, he is guaranteed to fail at least one.

Let us also suppose that we have compiled a table of classes and filled them with the number of complaints from students if two classes are in the same timeslot.

Naturally, we already have a guaranteed solution – put every final into its own timeslot. But, this is suboptimal since we will have a final exam period that lasts for months, so we need to put n some constraints.

- Suppose we have 15 timeslots and 70 classes, how do we assign a timeslot with a minimum number of complaints?

This is similar to graph coloring but in this case you're guaranteed to fail... To solve this, we can do the same thing we did with hill climbinmg.

- Randomly assign times and see if it increases complaints or decreases it
- How do we assign state? We could have a one dimensional array of classes where each element is the timeslot for that class (much simpler than two dimensional array and easier to perturb)
- Then we could perturb by picking a random class / element in the array and assigning a new random timeslot instead
- To determine cost, we just count all the classes with duplicate timeslots and the number of students in those classes

If we have a 2 dimensional array representing the compiled table of classes

```
nclass = 70
student = np.random.randint(0,100,(nclass, nclass))

np.random.seed(222)
for i in range(nclass-1):
    for j in range(i+1,nclass):
        student[i,j] = student[j,i]

for i in range(nclass):
    student[i,i] = 0

student[student>70] = 0

print(student)

plt.matshow(student)
```

we can find the number of students taking both class `i` and `j` using `student[i][j]`.

```python
n_class = 70
n_timeslot = 15

def cost(state):
    s = 0
    for c1 in range(n_class):
        for c2 in range(c1, n_class):
            if state[c1] == state[c2]:
                s+=student[c1, c2]
    return s

def purturb(state):
    class_to_change = np.random.randint(n_class)
    change_to = np.random.randint(n_timeslot)
    new_state = [x  for x in state]
    new_state[class_to_change] = change_to
    return new_state

def random_state():
    return np.random.randint(n_timeslot, size=n_class)

def better_time_slot(initial_state, n):
    current_state = initial_state
    current_complaints = cost(current_state)

    for i in range(n):
        to_try = purturb(current_state)
        try_score = cost(to_try)
        #print(to_try, try_score)
        if try_score < current_complaints:
            current_state = to_try
            current_complaints = try_score
            print(i, current_complaints)
    return current_state

    # profit!!!

better_time_slot(random_state(), 20000)
```

Substitution Cipher

We have probably heard about encryption methods like ROT13 before. ROT13 is in a subclass of ciphers known as *substitution* ciphers. It's one of the weakest encryption methods – basically having a map from one letter to another.

- Decryption is easy, we just do the reverse

As it turns out, we can use hill climbing to crack this encryption! We can sort of determine a score for a key to decrypt text, based on how readable the decrypted text is. However, we would need to have some way to keep the key as a state, perturb it and find the cost.

- The hardest part of this problem is how to tell a computer "how English" a sentence is
- We can cross reference with a dictionary, but if you have all letters right but one, it'll still get a score of 0.

Instead, we can look at the probability of one/two/three letters appearing in an English word

- 'a' and 'I' by themselves should appear more than 'g' and 'z'
- The bigram 'th' appears way more than 'di', so if the encryption key gives us a lot of 'th', it might be more likely to be correct than one that gives a lot of 'di

Luckily, Google has counted some of the common n-grams in thousands and thousands of books for us. Here's how we can score them

- For a one letter word, we will perform a lookup in the dictionary `unigram_score`
- For a two letter word, the score is based on `bigram_score`
- For words with three or more letters, the score is given by computing the score of every trigram in the word
- If the dictionary lookup fails, just give it a super negative score

```
import string
import math

cipher_text = "vxm enkv embgryjz vxrou ro vxm qnbzp r vxroi rk vxm roftrzrvl ny vxm xjefo er

def load_l2score_dict():
    with open('count_2l.txt') as f:
        total = 0
        d = {}
        for l in f:
            t, n = l.split()
            d[t] = int(n)
            total += int(n)
        for k,v in d.items():
            d[k] = math.log(d[k]/float(total))
        return d

def load_l3score_dict():
    with open('count_3l.txt') as f:
        total = 0
        d = {}
        for l in f:
            t, n = l.split()
            d[t] = int(n)
            total += int(n)
        for k,v in d.items():
            d[k] = math.log(d[k]/float(total))
    return d
unigram_score = {x: (math.log(0.5) if x=='a' or x=='i' else -100.) for x in string.ascii_low
bigram_score = load_l2score_dict()
trigram_score = load_l3score_dict()
```

```
def decrypt(msg, key): # key is decryption key
    return ''.join([key.get(s,s) for s in msg])

def token_score(token):
```

```python
    if len(token) == 1:
        return unigram_score.get(token, -100)
    elif len(token) == 2:
        return bigram_score.get(token, -100)
    else:
        s = 0
        for i in range(len(token)-2):
            trigram =  token[i:i+3]
            s+= trigram_score.get(trigram, -100)
        return s

def score(key):
    got = decrypt(cipher_text, key)
    # follow the rules above.
    s = 0
    for token in got.split(' '):
        s+= token_score(token)
    return s

def random_char():
    idx = np.random.randint(len(string.ascii_lowercase))
    return string.ascii_lowercase[idx]

def perturb(key):
    new_key = {k: v for k,v in key.items()}

    idx1 = random_char()
    idx2 = random_char()

    new_key[idx1] = key[idx2]
    new_key[idx2] = key[idx1]
    return new_key

def random_state():
    return {x:x for x in string.ascii_lowercase}

def hacker_man(initial_key, n):
    current_key = initial_key
    current_score = score(initial_key)

    for i in range(n):
        to_try = perturb(current_key)
        new_score = score(to_try)
        if new_score > current_score:
            current_score = new_score
            current_key = to_try
            print(current_score, i)
    print(decrypt(cipher_text, current_key))
    return current_key

hacker_man(random_state(), 10000)
```

This is much better than brute forcing it, which would require us to try 26! states.